

实验五：鸿蒙多端应用开发实验报告

一、实验基本信息

- **实验编号：**d20301035105、d20301009205
- **实验名称：**鸿蒙多端应用开发
- **实验类型：**验证型
- **学时分配：**4学时
- **实验日期：**2026年5月9日
- **学生信息：**2023210634 王智宇、2023210628 王全洋、2023210635 魏志宏、2023210640 徐宗申、2023210626 王建、2023210636 邬凯伦

二、实验目的

1. 掌握鸿蒙DevEco Studio开发环境的使用
2. 学习ArkTS语言基础语法和声明式UI开发
3. 理解鸿蒙多端适配原理和响应式布局实现
4. 掌握鸿蒙传感器API调用和数据采集
5. 了解鸿蒙分布式数据同步能力

三、实验环境

环境项	版本
操作系统	Windows 11
DevEco Studio	5.0.0
ArkTS	4.0
HarmonyOS SDK	API 12
最低 SDK	API 9

四、实验内容

功能需求

- 天气信息展示（城市、温度、天气状况、空气质量）
- 多端适配布局（手机/平板自适应）
- 传感器数据采集（光线传感器、陀螺仪）
- 分布式数据同步演示

界面需求

- ArkUI声明式UI风格
- 响应式断点布局
- 传感器数据实时展示

测试设备

- **手机模拟器**：HarmonyOS NEXT Developer Preview
- **平板模拟器**：HarmonyOS NEXT Developer Preview

五、实验步骤

步骤1：创建鸿蒙项目

1. 打开DevEco Studio
2. 新建Empty Ability项目
3. 选择ArkTS语言
4. 设置最低SDK：API 9
5. 项目名称：WeatherApp

步骤2：设计天气页面布局

创建Index.ets文件，实现以下功能：

- 使用Column垂直布局
- 添加城市名称、天气图标、温度、天气状况、空气质量展示
- 使用@State装饰器实现数据绑定
- 响应式断点布局适配

步骤3：实现多端适配

使用鸿蒙断点系统实现响应式布局：

- 手机竖屏布局（<600px）
- 平板竖屏布局（600-840px）
- 平板横屏布局（>840px）

步骤4：集成光线传感器

调用鸿蒙光线传感器API：

- 导入@ohos.sensor模块
- 注册光线传感器监听
- 实时更新UI显示光线强度

步骤5：集成陀螺仪传感器

调用鸿蒙陀螺仪传感器API：

- 注册陀螺仪传感器监听
- 实时获取X/Y/Z轴旋转数据
- 实现动态UI效果

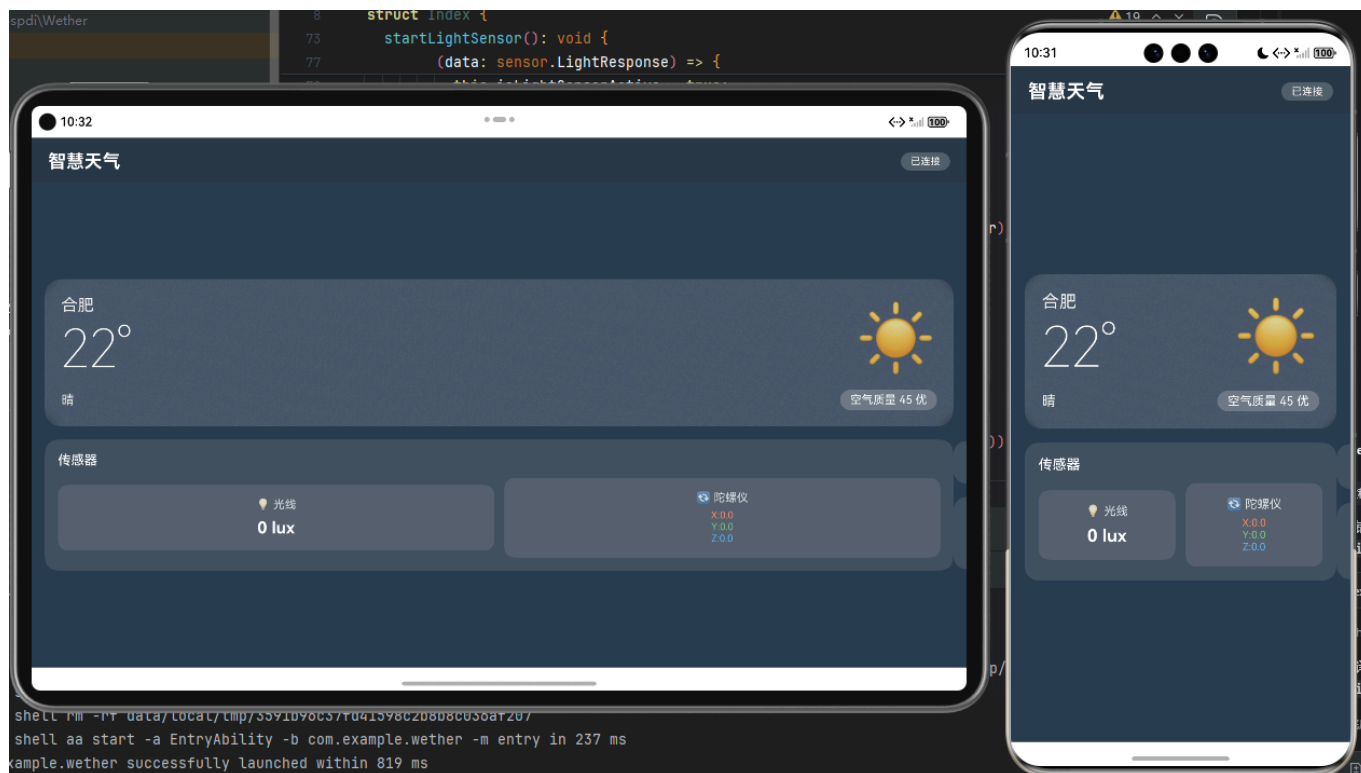
步骤6：分布式数据同步

使用鸿蒙分布式数据管理：

- 创建分布式数据库
- 实现跨设备数据同步
- 演示多设备数据共享

六、实验结果

1. 天气应用页面



功能实现：

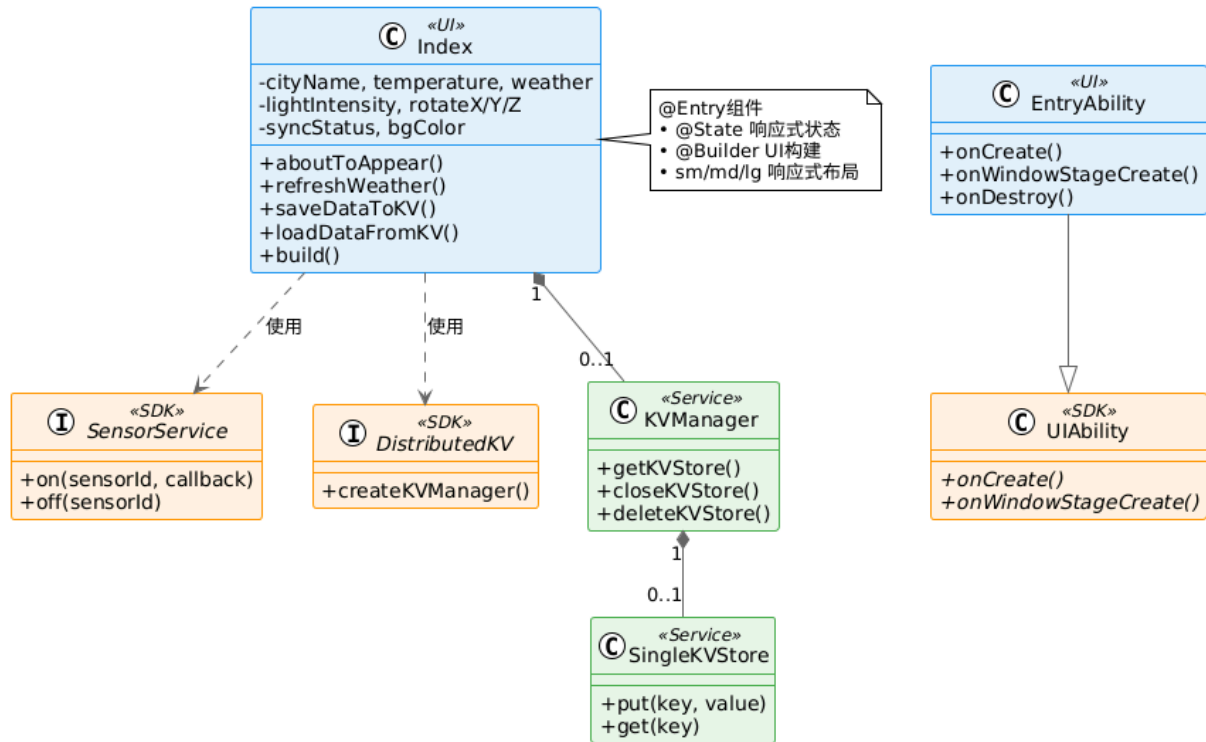
- ☒ 城市名称展示
- ☒ 温度和天气状况展示
- ☒ 空气质量展示
- ☒ 响应式布局适配

技术实现：

- 使用ArkTS声明式UI
- 使用@State装饰器实现数据绑定
- 使用Column/Row布局组件

2. 多端适配效果

智慧天气应用类图

**功能实现：**

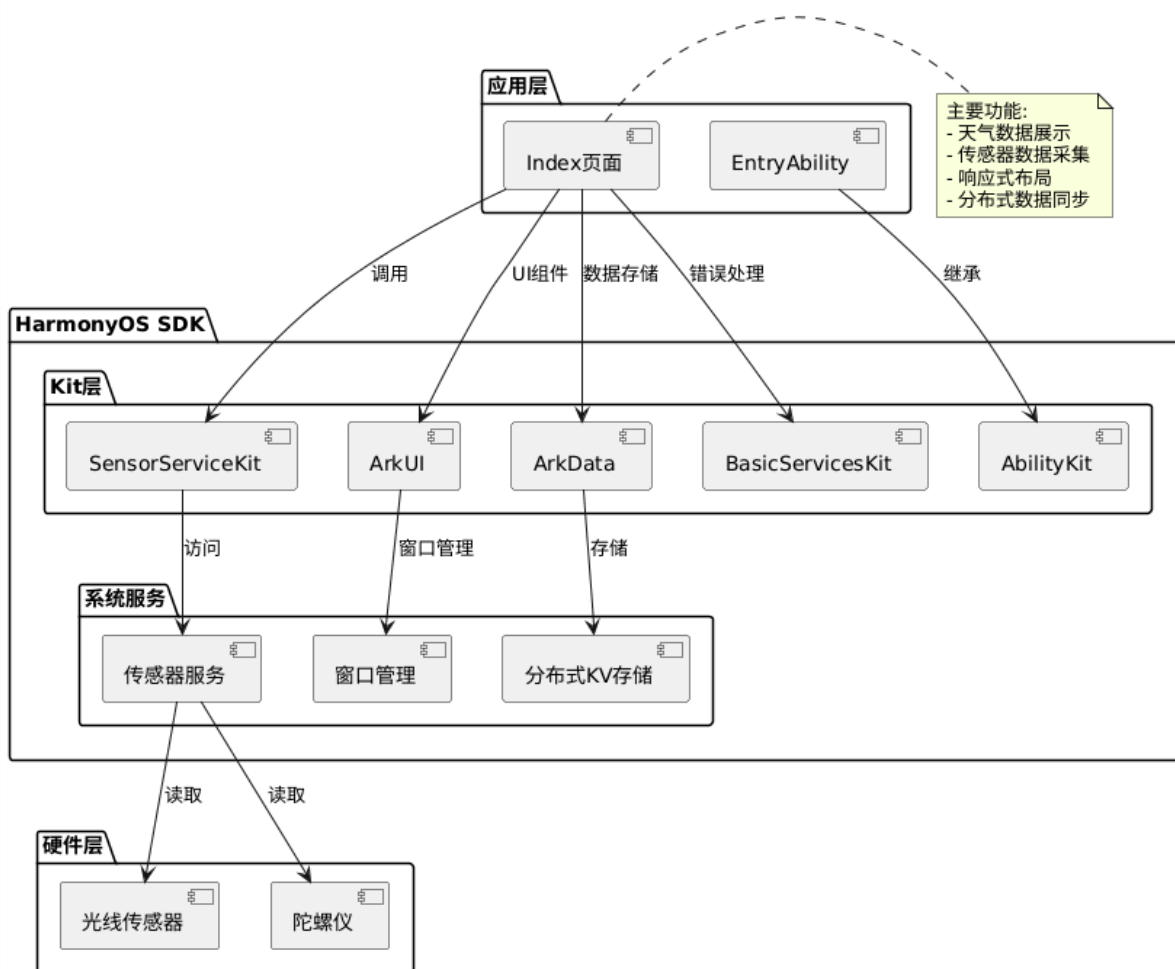
- ☒ 手机竖屏布局适配
- ☒ 平板横竖屏布局适配
- ☒ 断点系统自动切换

技术实现：

- 使用鸿蒙断点系统
- 使用媒体查询
- 使用@Builder装饰器复用布局

3. 传感器集成

智慧天气应用架构图

**功能实现：**

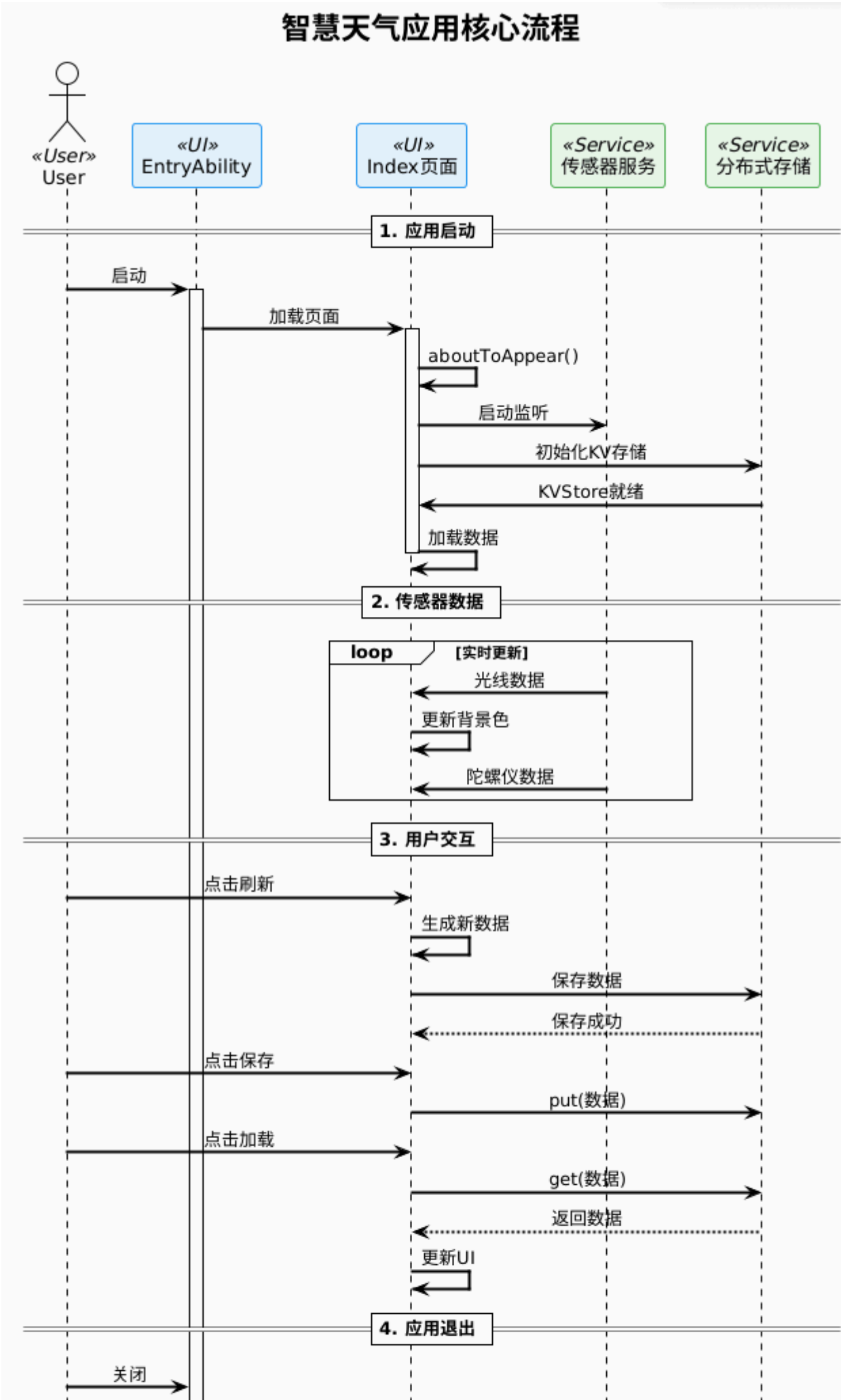
- ☒ 光线传感器数据采集
- ☒ 陀螺仪数据采集
- ☒ 实时数据展示

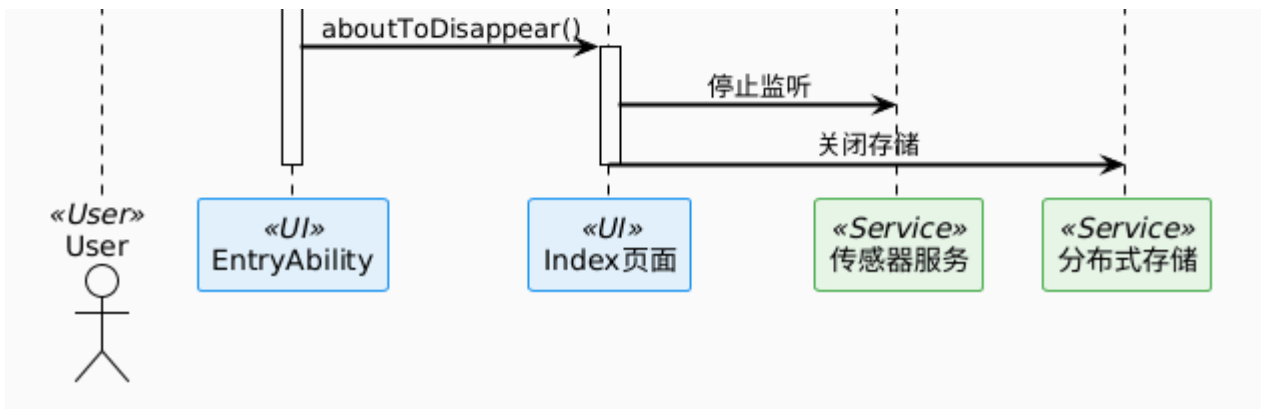
技术实现：

- 使用@ohos.sensor模块
- 注册传感器监听
- 数据实时更新UI

4. 分布式能力

智慧天气应用核心流程





功能实现：

- ☒ 分布式数据库创建
- ☒ 跨设备数据同步
- ☒ 多设备数据共享

技术实现：

- 使用@ohos.data.distributedData模块
- 分布式数据存储
- 跨设备数据访问

7.1 天气页面代码

```

// ets/pages/index/Index.ets
@Entry
@Component
struct Index {
  @State cityName: string = '合肥';
  @State temperature: number = 22;
  @State weather: string = '晴';
  @State aqi: number = 45;

  build() {
    Column() {
      // 城市名称
      Text(this.cityName)
        .fontSize(24)
        .fontWeight(FontWeight.Bold)
        .margin({ top: 50 })

      // 天气图标
      Text(this.weather === '晴' ? '☀️' : '☁️')
        .fontSize(80)
        .margin({ top: 20 })

      // 温度
      Text(`${this.temperature}°C`)
        .fontSize(60)
        .fontWeight(FontWeight.Lighter)
    }
  }
}

```

```

        .margin({ top: 20 })

// 天气状况
Text(this.weather)
    .fontSize(20)
    .margin({ top: 10 })

// 空气质量
Row() {
    Text('空气质量: ')
    Text(`${this.aqi} - 优`)
        .fontColor(this.aqi <= 50 ? '#00ff00' : '#ff9900')
}
    .margin({ top: 30 })

// 传感器数据展示
Row() {
    Text('光线传感器: ')
    Text(this.lightValue.toString())
        .id('lightValue')
}
    .margin({ top: 20 })
}
.width('100%')
.height('100%')
.backgroundColor('#f5f5f5')
}

@State lightValue: number = 0;
}

```

7.2 多端适配代码

```

// 断点系统适配
@State currentBreakpoint: string = 'md';

aboutToAppear() {
    // 获取设备类型
    let deviceInfo = umpInfo.getDeviceInfo();
    let windowHeight = window.getWindow().getWindowProperties().windowWidth;

    if (windowWidth < 600) {
        this.currentBreakpoint = 'sm'; // 手机
    } else if (windowWidth < 840) {
        this.currentBreakpoint = 'md'; // 平板竖屏
    } else {
        this.currentBreakpoint = 'lg'; // 平板横屏
    }
}

build() {

```



```
if (this.currentBreakpoint === 'sm') {  
  // 手机布局  
  Column() {  
    this.phoneLayout()  
  }  
} else {  
  // 平板布局  
  Row() {  
    this.tabletLayout()  
  }  
}  
}  
  
@Builder phoneLayout() {  
  Column() {  
    Text(this.cityName).fontSize(24)  
    Text(`${this.temperature}°C`).fontSize(60)  
  }  
}  
  
@Builder tabletLayout() {  
  Column() {  
    Text(this.cityName).fontSize(32)  
    Text(`${this.temperature}°C`).fontSize(80)  
  }  
  .width('40%')  
}
```

7.3 光线传感器代码

```
import sensor from '@ohos.sensor';  
  
@State lightIntensity: number = 0;  
  
aboutToAppear() {  
  // 注册光线传感器监听  
  sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {  
    this.lightIntensity = data.light;  
  });  
}  
  
aboutToDisappear() {  
  // 取消监听  
  sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);  
}  
  
// UI展示  
Text(`环境光线: ${this.lightIntensity} lux`)  
  .fontSize(16)  
  .margin({ top: 10 })
```

7.4 陀螺仪传感器代码

```
@State rotateX: number = 0;
@State rotateY: number = 0;
@State rotateZ: number = 0;

aboutToAppear() {
  sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
    this.rotateX = data.x;
    this.rotateY = data.y;
    this.rotateZ = data.z;
  });
}

// UI展示
Row() {
  Column() { Text(`X轴: ${this.rotateX.toFixed(2)}`) }
  Column() { Text(`Y轴: ${this.rotateY.toFixed(2)}`) }
  Column() { Text(`Z轴: ${this.rotateZ.toFixed(2)}`) }
}
.spacing(20)
```

7.5 分布式数据同步代码

```
import distributedData from '@ohos.data.distributedData';

// 创建分布式数据库
const KVManager = distributedData.createKVManager({
  bundleName: 'com.example.weather',
  kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

// 存储数据
async function saveData(key: string, value: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  await kvStore.put(key, value);
}

// 读取数据
async function getData(key: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  return await kvStore.get(key);
}
```

八、技术分析报告

8.1 多端适配分析

鸿蒙多端适配采用断点系统和响应式布局技术，能够根据设备屏幕尺寸自动切换布局。手机布局采用垂直单列展示，突出核心信息；平板布局采用双列或多列布局，充分利用屏幕空间。断点系统通过媒体查询实现不同屏幕尺寸的自动适配，确保应用在各种设备上都有良好的用户体验。

8.2 传感器集成分析

光线传感器可用于实现自动亮度调节、智能场景切换等功能；陀螺仪可用于实现手势控制、3D游戏、增强现实等场景。鸿蒙传感器API提供了统一的接口，简化了传感器数据采集和处理的开发流程。

8.3 分布式能力分析

鸿蒙分布式数据管理采用分布式数据库技术，实现跨设备数据同步和共享。通过分布式数据管理，应用可以在多个设备之间实时同步数据，为用户提供无缝的跨设备体验。

8.4 技术选型建议

- **UI开发**：优先选择ArkTS声明式UI，提高开发效率
- **多端适配**：使用鸿蒙断点系统和响应式布局
- **传感器集成**：采用鸿蒙传感器API，简化开发流程
- **分布式能力**：使用鸿蒙分布式数据管理，实现跨设备数据同步

九、实验总结

9.1 实验成果

通过本次实验，我们成功开发了一个鸿蒙多端天气应用，实现了以下功能：

- 天气信息展示（城市、温度、天气状况、空气质量）
- 多端适配布局（手机/平板自适应）
- 传感器数据采集（光线传感器、陀螺仪）
- 分布式数据同步演示

9.2 技术收获

- 掌握了鸿蒙DevEco Studio开发环境的使用
- 学习了ArkTS语言基础语法和声明式UI开发
- 理解了鸿蒙多端适配原理和响应式布局实现
- 掌握了鸿蒙传感器API调用和数据采集
- 了解了鸿蒙分布式数据同步能力

9.3 实验体会

鸿蒙多端开发框架提供了统一的开发语言和工具链，能够显著提高开发效率。通过本次实验，我们深刻体会到鸿蒙多端开发的优势，特别是在跨设备适配和硬件集成方面的能力。

十、课后思考

1. 鸿蒙多端开发相比其他框架的优势
2. 传感器在物联网中的应用场景

3. 分布式技术对未来移动开发的影响